

PostgreSQL, JPA & Hibernate Fundamentals

Complete detailed study notes • Beginner-friendly • Book Catalog project

DAY 6 GOAL

Understand how your Spring Boot application stores real data in PostgreSQL, how Java objects become database rows, what JPA and Hibernate actually do, and how repositories, relationships, loading and database migrations fit together.

What you will learn today

- PostgreSQL and relational database basics
- Tables, rows, columns, primary keys, foreign keys and indexes
- SQL basics needed for backend development
- JPA vs Hibernate vs Spring Data JPA
- Entities, IDs and repositories
- Persistence context and how save/find really work
- Dirty checking and transactions
- Lazy vs eager loading
- Book–Author relationships
- Database migrations and generated SQL
- Testing and verifying persistence behavior

MENTAL MODEL

PostgreSQL is where the data lives. JPA is a standard way for Java code to describe database work. Hibernate is a popular implementation of that standard. Spring Data JPA gives you convenient repository methods so you write less database plumbing.

Important: do not try to memorize annotations first. First understand what is happening to the data.

INDEX — COMPLETE TOPICS

All topics covered in these notes. The numbered headings are kept in sequence and topics flow naturally across pages.

1. What is a database and why backend applications need one?
2. PostgreSQL: the database used in this project
3. Relational database basics: tables, rows and columns
4. Primary keys and why every row needs an identity
5. Foreign keys and relationships between tables
6. Indexes: making lookups faster
7. SQL basics you should know
8. How our Book Catalog data is represented
9. JDBC, JPA, Hibernate and Spring Data JPA — what is the difference?
10. What is an Entity?
11. The @Entity and @Id annotations
12. Generated IDs and ID strategies
13. Mapping Java fields to database columns
14. Repositories and JpaRepository
15. What actually happens when save() is called?
16. What actually happens when findById() is called?
17. The persistence context — Hibernate's working area
18. Dirty checking
19. Transactions and why they matter
20. Entity states: new, managed, detached and removed
21. One-to-many and many-to-one relationships
22. Book–Author relationship for the project
23. Lazy loading
24. Eager loading and why it can hurt
25. The N+1 query problem
26. Cascade and orphan removal — basic understanding
27. Database configuration in Spring Boot
28. Connecting Book Catalog to PostgreSQL
29. Schema generation vs database migrations
30. Flyway migrations — why production needs them
31. Reading and verifying generated SQL
32. Testing persistence behavior
33. Common beginner mistakes
34. Complete Book Catalog persistence implementation
35. Practice / interview questions
36. End-of-day self-check

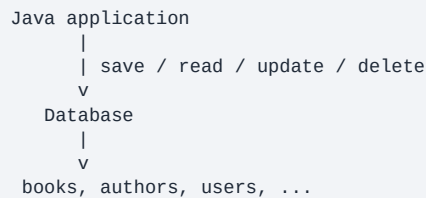
TARGET OUTCOME

By the end of Day 6, you should be able to explain the path Java object → JPA/Hibernate → SQL → PostgreSQL → result back to Java, and implement the Book and Author persistence layer yourself.

1 What is a database and why backend applications need one?

A database is simply a place where an application keeps information so that it is still there after the application stops running.

If your Book Catalog stores books only in a Java ArrayList, all the books disappear when the application restarts. A database keeps them on disk.



A real backend usually needs reliable storage because many requests can arrive, data must survive restarts, and multiple users may work with the same information.

TODAY'S GOAL

Replace temporary in-memory book storage with PostgreSQL-backed storage.

2 PostgreSQL: the database used in this project

PostgreSQL is a relational database. It stores structured data in tables and lets applications communicate with it using SQL.

For this roadmap, PostgreSQL will store our Book Catalog data. You do not need to become a database administrator today; focus on the concepts a Java backend developer uses every day.

Important ideas: database → schema → table → row → column.

```
CREATE TABLE books (
  id BIGSERIAL PRIMARY KEY,
  title VARCHAR(200) NOT NULL,
  price NUMERIC(10,2) NOT NULL
);
```

You can think of a table as a structured spreadsheet, but PostgreSQL provides rules, relationships, transactions, concurrency control and powerful querying.

3 Relational database basics: tables, rows and columns

A **table** holds one kind of data. A **row** represents one record. A **column** describes one property of that record.

```
books
+-----+-----+-----+
| id | title           | price |
+-----+-----+-----+
| 1  | Clean Code      | 35.00 |
| 2  | Effective Java  | 42.00 |
+-----+-----+-----+
```

In Java, the same information might be represented by a Book object. The database stores the persistent version of that information.

REMEMBER

Java object ≠ database row, but JPA/Hibernate can map between the two.

4 Primary keys and why every row needs an identity

A primary key is a column, or set of columns, used to uniquely identify a row.

For our Book table, id is a natural choice. Two different books should not accidentally have the same id.

```
id = 1 -> one specific book
id = 2 -> another book
id = 3 -> another book
```

Your API URL often uses this identity: GET /api/books/2 means “find the book whose id is 2.”

A primary key also gives the database a clear way to reference a specific record when updating or deleting it.

5 Foreign keys and relationships between tables

A foreign key connects one table to another. For example, a book can refer to an author.

```
authors
+-----+
| id | name      |
+-----+
| 10 | Robert Martin|
+-----+

books
+-----+-----+
| id | title      | author_id|
+-----+-----+
| 1  | Clean Code | 10       |
+-----+-----+
```

author_id points to authors.id. The database can use this relationship to prevent invalid references.

SIMPLE RULE

Primary key identifies a row. Foreign key points from one table to a row in another table.

6 Indexes: making lookups faster

An index is an extra data structure that helps the database find rows faster for certain searches.

Without a useful index, PostgreSQL may need to inspect many rows. With a suitable index, it can often locate matching data much faster.

```
CREATE INDEX idx_books_title
ON books(title);
```

Indexes are not free. They use storage and make inserts/updates slightly more expensive because the index also has to be maintained.

BACKEND DEVELOPER RULE

Do not add an index to every column. Add indexes for important search, join, uniqueness or ordering patterns.

7 SQL basics you should know

Even when using JPA, you should understand basic SQL. Hibernate ultimately communicates with the database using SQL.

```
SELECT id, title
FROM books
WHERE price > 30
ORDER BY title;

INSERT INTO books (title, price)
VALUES ('Clean Code', 35.00);

UPDATE books
SET price = 38.00
WHERE id = 1;

DELETE FROM books
WHERE id = 1;
```

Know the basic operations: SELECT, INSERT, UPDATE, DELETE, WHERE, ORDER BY, JOIN and basic aggregate functions.

You do not need to write every query manually in a Spring Data JPA application, but you should be able to read the SQL Hibernate generates.

8 How our Book Catalog data is represented

We will have at least two main pieces of data: **Book** and **Author**. The exact relationship depends on the business rule. For learning, use one Author with many Books.

```
Author
  id
  name

Book
  id
  title
  isbn
  price
  author_id
```

This means one author can be linked to many books, while each book has one author in our simplified model.

DESIGN FIRST

Decide what the data means before writing annotations. Annotations should describe the model, not replace thinking about it.

9 JDBC, JPA, Hibernate and Spring Data JPA — what is the difference?

JDBC is a low-level Java way to talk to a relational database. You write more SQL and handle more database details yourself.

JPA is a Java standard that defines how Java objects can be mapped to relational data. It is a specification, not the database itself.

Hibernate is a popular implementation of JPA. It does the actual work of translating object operations into SQL and managing the persistence context.

Spring Data JPA sits on top and makes repository code much easier to write.

```
Your code
  ↓
Spring Data JPA
  ↓
JPA API
  ↓
Hibernate
  ↓
JDBC / database driver
  ↓
PostgreSQL
```

MEMORIZE THIS ORDER

Spring Data JPA is the convenience layer; Hibernate is the common JPA implementation; PostgreSQL is the actual database.

10 What is an Entity?

An Entity is a Java class whose objects are meant to be stored in the database.

```
@Entity
public class Book {

    @Id
    @GeneratedValue
    private Long id;

    private String title;
    private BigDecimal price;
}
```

The class describes the shape of the Java object. Hibernate uses that information to understand how it should represent the object in database tables.

An entity is not automatically the same thing as an API request/response DTO. Keep that separation from Day 2 in mind.

11 The @Entity and @Id annotations

@Entity tells JPA/Hibernate that this class should participate in persistence.

@Id tells it which field is the identity of each entity.

```
@Entity
public class Author {

    @Id
    private Long id;

    private String name;
}
```

Every entity needs an identifier. Without a clear identity, Hibernate cannot reliably track which database row a Java object represents.

12 Generated IDs and ID strategies

You normally do not want your application manually inventing database IDs. Let the database or JPA strategy generate them.

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
```

The exact strategy can depend on the database and project requirements. For a beginner PostgreSQL project, understand the idea first: a new entity gets an identity when it is persisted.

INTERVIEW IDEA

Know the difference between an application-assigned ID and a database-generated ID.

13 Mapping Java fields to database columns

By default, Hibernate can map simple Java fields to columns. You can also control the mapping explicitly.

```
@Column(nullable = false, length = 200)
private String title;

@Column(nullable = false, precision = 10, scale = 2)
private BigDecimal price;
```

Use explicit constraints when they represent real business/database rules. For example, a book title should not normally be null.

Also understand that Java validation and database constraints solve different problems: validation gives a friendly API-level response; the database remains the final data integrity boundary.

14 Repositories and JpaRepository

A repository is the part of your application that deals with storing and retrieving data. Spring Data JPA can create common operations for you.

```
public interface BookRepository
    extends JpaRepository<Book, Long> {
}
```

You immediately get methods such as `save()`, `findById()`, `findAll()`, `deleteById()` and `existsById()`.

This removes repetitive CRUD code. You can also define query methods for common searches.

```
List<Book> findByTitleContainingIgnoreCase(
    String title
);
```

THINK OF IT AS

A ready-made door between your Service layer and the database.

16 What actually happens when `save()` is called?

When you call `repository.save(book)`, Spring Data JPA delegates to the JPA `EntityManager`. Hibernate then decides whether it needs to insert a new row or update an existing one.

```
Book book = new Book();
book.setTitle("Clean Code");

Book saved = bookRepository.save(book);
```

For a new entity, Hibernate will generally schedule an INSERT. The SQL may not execute at the exact line of `save()`; transaction/flush behavior matters.

The key beginner lesson: **`save()` does not mean “send SQL immediately right now.”** Hibernate manages when SQL is synchronized with the database.

16 What actually happens when `findById()` is called?

`findById(id)` asks the repository for an entity with that identity. Hibernate can check its persistence context first, and if the entity is not already available there, it can query PostgreSQL.

```
Optional<Book> result =
    bookRepository.findById(1L);

Book book = result.orElseThrow(
    () -> new ResourceNotFoundException("Book not found")
);
```

The returned `Book` is an object that Hibernate is tracking while it is managed by the current persistence context.

17 The persistence context — Hibernate's working area

The persistence context is a set of entity objects that Hibernate is currently tracking for a transaction/session.

```
Database
  ↑ ↓
Hibernate
  ↑ ↓
Persistence Context
  ↑
Java entity objects
```

If Hibernate is tracking a `Book` and you change its title, Hibernate can notice the change and later generate an UPDATE.

EASY ANALOGY

Think of the persistence context as Hibernate's working desk. It knows which entity objects are currently being watched and what their original state was.

Dirty checking

Dirty checking means Hibernate notices changes made to managed entities and can update the database without you explicitly calling an update method.

```
@Transactional
public void changeTitle(Long id) {
    Book book = repository.findById(id)
        .orElseThrow();

    book.setTitle("New Title");

    // No explicit update(book) required.
}
```

When the transaction is flushed/committed, Hibernate can detect the changed field and issue an UPDATE.

IMPORTANT

Dirty checking works for managed entities. It is not magic for every arbitrary Java object.

Transactions and why they matter

A transaction groups database work into one logical unit. Either the work completes successfully, or the database can roll it back when a failure occurs.

```
@Transactional
public void updateBook(...) {
    // read
    // change
    // other database work
    // commit as one unit
}
```

For a simple read, a transaction may be handled for you by the framework. For business operations that change data, understanding transaction boundaries is essential.

Transactions also explain why SQL may be delayed until flush or commit.

Entity states: new, managed, detached and removed

An entity can move through different states during its life.

```
NEW
|
| persist/save
v
MANAGED
|
| detach / context ends
v
DETACHED

MANAGED
|
| remove
v
REMOVED
```

New: not yet associated with the persistence context. **Managed:** Hibernate tracks it. **Detached:** the context no longer tracks it. **Removed:** marked for deletion.

You do not need to memorize every edge case today. You need to understand why changing a managed object can cause a database update.

One-to-many and many-to-one relationships

A one-to-many relationship means one record can be associated with many records. A many-to-one relationship is the same relationship viewed from the other side.

```
Author 1
|
+---- Book A
|
+---- Book B
|
+---- Book C
```

In JPA, the Book side commonly owns the foreign key using `@ManyToOne`, while Author can expose the collection using `@OneToMany`.

```
@ManyToOne
@JoinColumn(name = "author_id")
private Author author;
```

The exact mapping should match the database relationship you want.

22 Book–Author relationship for the project

For this learning project, use one Author → many Books. The books table contains `author_id`.

```
@Entity
class Author {
    @Id
    @GeneratedValue
    private Long id;

    private String name;

    @OneToMany(mappedBy = "author")
    private List<Book> books = new ArrayList<>();
}

@Entity
class Book {
    @Id
    @GeneratedValue
    private Long id;

    private String title;

    @ManyToOne
    @JoinColumn(name = "author_id")
    private Author author;
}
```

The **mappedBy** value points to the field on Book that owns the relationship. Do not blindly copy this; understand which table actually contains the foreign key.

23 Lazy loading

Lazy loading means related data is not loaded until you actually ask for it.

```
Book book = repository.findById(1L).orElseThrow();

// Author may be loaded only when needed:
String name = book.getAuthor().getName();
```

This can save unnecessary database work when the application does not need the related object.

THINK OF IT

Lazy = “I will fetch that related data when I need it.”

24 Eager loading and why it can hurt

Eager loading means related data is loaded along with the entity. This can look convenient, but it can create unnecessary work.

If you load 100 books and each book eagerly loads an author, Hibernate may fetch a lot more data than the endpoint actually needs.

The bigger problem appears when several relationships are eager or when collections become large.

RULE OF THUMB

Do not choose EAGER simply because it makes an error disappear. Decide what data the use case actually needs.

25 The N+1 query problem

N+1 means the application runs one query to load a list and then another query for each item to load related data.

```
1 query: SELECT books ...
+ N queries: SELECT author ... for each book

Total = N + 1 queries
```

For 100 books, this can become 101 database queries. That may work in development and become painfully slow in production.

Learn to watch the generated SQL. Later you will learn tools such as fetch joins, entity graphs and carefully designed DTO queries to solve specific cases.

26 Cascade and orphan removal — basic understanding

Cascade controls whether certain operations performed on one entity are propagated to related entities.

For example, cascading persist can make saving an Author also persist newly created Books. This is powerful but should be used only when the relationship's ownership and lifecycle really justify it.

orphanRemoval is about removing a child entity when it is removed from the parent's relationship collection, depending on the mapping and operation.

CAUTION

Do not add cascade settings just because they make code shorter. Cascading can delete or persist more data than you intended.

27 Database configuration in Spring Boot

Your application needs to know how to connect to PostgreSQL. Keep this configuration outside Java code.

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/book_catalog
    username: ${DB_USERNAME}
    password: ${DB_PASSWORD}

  jpa:
    show-sql: true
    hibernate:
      ddl-auto: validate
```

The exact settings depend on your project. The important habit is to separate environment-specific configuration and never commit real production passwords.

28 Connecting Book Catalog to PostgreSQL

The practical setup is: create a PostgreSQL database, add the PostgreSQL JDBC driver, configure the datasource, configure JPA/Hibernate, then start the application and verify the connection.

```
CREATE DATABASE book_catalog;
```

In Maven, the application needs the PostgreSQL driver plus Spring Data JPA. Spring Boot then creates the infrastructure needed to communicate with the database.

VERIFY

Do not assume the connection works because the application starts. Check the logs, connect to PostgreSQL, and inspect the tables/schema.

29 Schema generation vs database migrations

Hibernate can generate or modify database tables based on entity classes. This is useful for learning and local experiments, but it is not a good production change-management strategy.

A migration is a versioned database change such as “V2 adds the isbn column.” The migration becomes part of the project's history.

```
V1__create_books.sql
V2__create_authors.sql
V3__add_isbn_to_books.sql
```

PRODUCTION HABIT

Database structure should change through reviewed, repeatable migrations—not by manually editing a production database.

30 Flyway migrations — why production needs them

Flyway is a migration tool commonly used with Spring Boot. It runs versioned SQL scripts in order and records which migrations have already been applied.

```
src/main/resources/db/migration/
  V1__create_authors.sql
  V2__create_books.sql
  V3__add_isbn.sql
```

This means another developer, a CI server, staging and production can apply the same database changes in a predictable sequence.

Learn the naming pattern, ordering, idempotent thinking, rollback considerations and the importance of reviewing migration SQL before it reaches production.

31 Reading and verifying generated SQL

Turn on SQL logging in a local environment and watch what Hibernate actually sends to PostgreSQL.

```
repository.findById(1L);

-- conceptually you may see:
select
  b.id,
  b.title,
  b.price,
  b.author_id
from books b
where b.id = ?;
```

Generated SQL is one of your best debugging tools. If an endpoint is slow, the SQL often tells you why: too many queries, missing indexes, unnecessary joins or unexpectedly large results.

DEVELOPER HABIT

Do not treat Hibernate as magic. Learn to inspect the SQL behind your Java code.

32 Testing persistence behavior

Persistence tests verify that your application actually stores and retrieves data correctly. Do not rely only on controller tests.

Important cases: save a book, read it back, update it, delete it, find a missing book, verify the Book–Author relationship, and verify constraints.

```
@Test
void savesAndFindsBook() {
    Book saved = repository.save(book);

    Optional<Book> result =
        repository.findById(saved.getId());

    assertTrue(result.isPresent());
}
```

For database integration tests, use a real PostgreSQL-compatible environment when practical. Testcontainers is a common option later in your roadmap.

33 Common beginner mistakes

1. Thinking JPA is the database. It is not.
2. Thinking save() always executes SQL immediately.
3. Making every relationship EAGER.
4. Returning entities directly from every API endpoint.
5. Ignoring generated SQL.
6. Changing production tables manually.
7. Storing database passwords in Git.
8. Adding indexes without understanding the query pattern.
9. Using cascade without understanding ownership.
10. Assuming tests that mock repositories prove database behavior.

FIX

For every persistence feature, ask: What is the table? What is the key? What SQL should happen? What happens inside a transaction? How will I test it?

34 Complete Book Catalog persistence implementation

Your Day 6 implementation should connect the work from the previous days to real PostgreSQL storage.

- Step 1:** Add Spring Data JPA and PostgreSQL dependencies.
- Step 2:** Create the PostgreSQL database.
- Step 3:** Configure the datasource through environment/configuration.
- Step 4:** Create Author and Book entities.
- Step 5:** Add primary keys and the Book → Author relationship.
- Step 6:** Create BookRepository and AuthorRepository.
- Step 7:** Update the Service layer to use repositories instead of in-memory lists.
- Step 8:** Add migrations for the schema.
- Step 9:** Run the application and inspect the generated SQL.
- Step 10:** Test create, read, update, delete and failure cases.

```
Controller
↓
Service
↓
Repository
↓
Spring Data JPA
↓
Hibernate
↓
JDBC driver
↓
PostgreSQL
```

DONE MEANS

You can stop PostgreSQL, restart your Spring Boot application, and your previously saved books are still there.

Database basics

1. What is a relational database?
2. What is a table, row and column?
3. What is a primary key?
4. What is a foreign key?
5. Why do we use indexes?
6. What is the difference between an index and a primary key?

JPA / Hibernate

7. What is JPA?
8. What is Hibernate?
9. What is Spring Data JPA?
10. JPA vs Hibernate?
11. What does `@Entity` do?
12. Why is `@Id` required?
13. What happens when `repository.save()` is called?
14. What is the persistence context?
15. What is dirty checking?
16. What is a transaction?

Relationships / performance

17. What is `@ManyToOne`?
18. What is `@OneToMany`?
19. Where should the foreign key live in the Book–Author model?
20. What is lazy loading?
21. Why can eager loading be dangerous?
22. What is the N+1 query problem?
23. What is cascade? When would you avoid it?

Production thinking

24. Why should database passwords not be committed to Git?
25. Why are migrations better than manually editing production tables?
26. Why should you inspect generated SQL?
27. How would you test that a Book really persisted to PostgreSQL?
28. What could make a simple `findById()` unexpectedly slow?

CHALLENGE

Without looking at your project, draw the complete path from `POST /api/books` to the PostgreSQL `INSERT`. Then explain where validation, Service logic, repository code, Hibernate and the transaction fit.

End-of-day self-check

Before moving to Day 7, you should be able to answer these without notes:

- I can explain PostgreSQL, tables, rows, primary keys and foreign keys.
- I can explain JPA vs Hibernate vs Spring Data JPA.
- I can create a basic `@Entity` with an `@Id`.
- I understand what a repository does.
- I can explain `save()`, `findById()` and dirty checking at a high level.
- I understand the purpose of transactions.
- I understand Book–Author relationships.
- I can explain lazy loading and the N+1 problem.
- I connected the Book Catalog to PostgreSQL.
- I used migrations instead of manually changing the schema.
- I inspected the SQL produced by Hibernate.
- I wrote tests for persistence behavior.

FINAL TEST

Close the PDF. Open your project. Build the Book and Author persistence layer from memory. If you get stuck, identify the exact concept you forgot, learn it, and continue rather than copying the solution.